- A ve B matrisler olsun. $A=\begin{bmatrix}1&2\\3&4\end{bmatrix}$ ve $B=\begin{bmatrix}2&2\\2&2\end{bmatrix}$ Bu iki matris için :
- A == B iki matris içinde karşılık gelen indexleri kıyaslar aynı olanlar 1 farklı olanlar 0 döner. $A==B \begin{bmatrix}0&1\\0&0\end{bmatrix}$
- A > 2 her elemanı 2 ile kıyaslar 2'den büyük olanlar 1 küçük olanlar 0 döner. $\begin{bmatrix}0&0\\1&1\end{bmatrix}$

↳ Eğer iki matris eşit mi diye kontrol etmek istersek isequal(A,B) ; 0 döner.

- isinf(A) A matrisi içinde sonsuz (±Inf) var mı? onu kontrol eder. $\begin{bmatrix}0&0\\0&0\end{bmatrix}$
- isnan(A) A matrisi içinde notanumber(NaN) var mı? onu kontrol eder. $C=\begin{bmatrix}NaN&1\\3&7\end{bmatrix}$ isnan(C) $=\begin{bmatrix}1&0\\0&0\end{bmatrix}$
- NaN == NaN cevabı 0 (sıfır) a eşittir.
- complex(a,b) sonuç olarak a+bi döner. complex(1,2) = 1+2i
- & and | or ~ not x=5; ~(x>0) sonuç 0 çıkar xor(A,B) sadece bir tanesi 'true' ise 1 döner.

00 → 0
01 → 1
10 → 1
11 → 0

- Eğer tek bir sonuç çıkan bir sorgu ise & yerine && kullanılabilir. Nasıl yani?
- x=[1 2 3], y=0; x>3 & y>3 en az 3 adet sonuç döner.
- a=2, b=3; a>0 & b>0 1 adet değer döner. Burda && kullanılabilir.
- Aynı problem or'da da geçerli tek sonuç döncekse || çift çizgi kullanılabilir.
- all() fonksiyonu içine giren elemanları 0 (sıfır) dan farklı mı diye kontrol eder. Lakin bir dizi girerse içinde bir tane bile 0 olsa sonuç=0

↳ a=[1 2 3] b=[3 0 2] all(a) =1 all(b) =0

- any() içine aldığı dizide bir tane eleman bile sıfırdan farklıysa 1 döner: any(a)→1, any(b)→1
- all(x>0) bütün elemanlar pozitif mi? any(x>0) pozitif bir tane de olsa eleman var mı?
- $A=\begin{bmatrix}1&2\\3&4\end{bmatrix}$ $B=\begin{bmatrix}1&4\\3&5\end{bmatrix}$ all ve any matrislerde sütun bazlı değerlendirme yapar: all(A==B) 1 0
- any(A==B) 1 1 neden? ~~1/3 = 1/3~~ / 2/4 2/5 en azından 2'ler eşit bu sebepten 1 1

find() fonksiyonun içinde eğer koşul verilmezse 0 (sıfır) dan farklı elemanların indexlerini döndürür.

- Eğer koşul verilirse koşulu sağlayan elemanların index lerini döndürür.
- X = [1 0 -INF 7]
- **find(x) = 1 3 4** eğer a = find(x), **x(a)** dersek bize o elemanları verir. 8 1 -INF 7 ters fonksiyon almak gibi
- Koşulumuz x<0 olsa f = find(x<0) = 3 x(f) = -INF

isfinite() fonksiyonu içine aldıklarından sonlu elemanları seçer. (±INF ve NaN hariçi) X = [1 0 -INF 7] isfinite(x) = 1 1 0 1

- Matrislerde find() koşulu sağlayan elemanların Index nosunu verir. $\begin{matrix}1\\3\\5\end{matrix}$ gibi.
- X = [1 0 -INF 7] logical(x) = 1 0 1 1 0'lar false 0 kalır, diğerleri 1
- a = true, b = false, c = true için a+b+c = 2 1+0+1 a*2 + b - 5*c = 1.2 +0 −5.1 = −3,1

↳ logical sınıfta bölme işlemi yapamazsın.

if -end ifden sonra koşul gelir sağlıyorsa çalışacak komut ve kapanış, end.

```
x=10;  y=15;   if y>x    disp('x küçük'); end
```

- Else - Elseif biliyorsun zaten farklı birşey yok.

```
x=10;  y=15;   if y>x    disp('x küçük')
               elseif x==y  disp('x ve y eşit')
               else ('x büyük')   end
```

For Döngüsü for i = başlangıç : adım : bitiş → komut → end

- eğer i = 10:20 denirse 10 11 12 ... 20 olur.

```
for x=[pi/6 pi/4 pi/3]
    disp([x, sin(x)])  end
```

- direkt x = [a b c] verilirse a ardından b ve son c denenir.

- Eğer matris yazdırmak istersek 8 for x = A disp(x) end
  fakat $\begin{bmatrix}1&2\\3&4\end{bmatrix}$ olan matris $\begin{matrix}1\\3\\2\\4\end{matrix}$ çıktısı verir. Index sırası ü

While Döngüsü

while - koşul - komut - end

- Koşul doğru olduğu sürece komut tekrar eder. Bu sebepten for'un aksine bir sayaç şarttır.
  - i<5 dersek şarta bu i'nin her döngüde bir miktar ↑artış göstermesi için while - şart - komut - artış - end olmalı.

```
i=5; while i<=5 disp(i) i=i+1; end
     while  koşul  komut  artış  bitiş
```

- Bu döngülere erken bitiş ihtimalleri göz önüne alınarak koşullu çıkışlar eklenebilir. Bu yapı If-Break komutlarıdır.

while → şart → [if → şart → break → end] → komut → artış → end

```
i=13; while i<98
        if isprime(i)
            disp('sayı asal') end
        disp('asala kadar devam')
        i=i+3; end
```

Switc-Case

amaç ve hizmetini biliyoruz. Yapısı ⁘

switch → ifade → case → 1.değer → komut
→ case → 2.değer → komut
→ otherwise // else misali
komut
end

```
gun=3;  switch gun  case 1  disp('one')
                    case 2  disp('two')
                    case 3  disp('three')
                    otherwise  disp('four....') end
```